

DocEngine — NexoraDB Core

API

مستندات کامل فنی

0.1.0 (MVP)	نسخه
Query · GraphEngine · Cython/Python Bindings	تیم مخاطب
core/DocEngine.h · core/DocEngine.cpp · CMakeLists.txt	فایل های مرتبط

فهرست مطالب

۱. معماری و جایگاه DocEngine
۲. ساختارهای داده مشترک
۳. چرخه حیات
۴. مدیریت Schema و Collection
۵. مدیریت Index
۶. مدیریت Foreign Key
۷. عملیات CRUD
۸. Internal API برای GraphEngine
۹. LookupJoin
۱۰. مدیریت Transaction

۱۱. توابع کمکی

۱۲. راهنمای تیم Query (UpdateSpec و Condition)

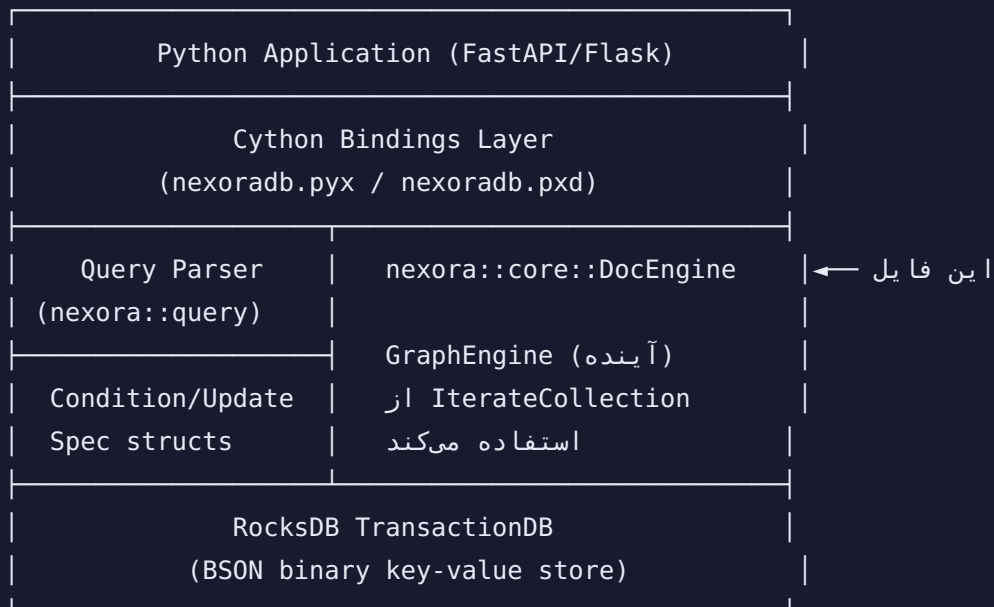
۱۳. راهنمای تیم GraphEngine

۱۴. راهنمای تیم Cython/Python Bindings

۱۵. فرمت کلیدهای RocksDB

۱۶. محدودیت‌ها و TODO های آینده

۱. معماری و جایگاه DocEngine



مسیر یک Query کامل

1. Python: `db.find("users", {"age": {"$gt": 25}})` کاربر
2. Cython: `dict` به `Condition struct` (بدون JSON بدون) (انتقال JSON)
3. Cython: `engine.FindMany("users", condition, limit, skip)` فراخوانی
4. DocEngine: `Index` بررسی `IndexScan` اگر وجود دارد $\rightarrow$ `FullScan` در غیر این صورت
5. DocEngine: `MatchesCondition` روی هر سند BSON
6. DocEngine: آرایه اسناد JSON با `DBResult` بازگشت
7. Cython: `Python list of dicts` تبدیل به

۲. ساختارهای داده مشترک

DBResult

خروجی استاندارد تمام توابع عمومی DocEngine.

```
struct DBResult {
    bool success = false;
    std::string data = ""; // مقدار بازگشتی (BSON/JSON/ID/count)
    std::string error_msg = ""; // پیام خطا در صورت failure

    static DBResult Ok(std::string payload = "");
    static DBResult Err(std::string msg);
};
```

حالت	success	data	error_msg
موفق InsertOne	true	"uuid-v4-string"	" "
موفق FindById	true	محتوای BSON	" "
موفق Count	true	"42"	" "
خطا	false	" "	توضیح خطا

SchemaDefinition

```

struct SchemaField {
    std::string name;
    FieldType type;          // String, Int32, Int64, Float64, Bool, Array, Object
    bool required;
    bool unique;
    std::optional<std::string> default_val;
};

struct SchemaDefinition {
    std::vector<SchemaField> fields;
    bool strict = false; // reject فیلدهای اضافه true: اگر
};

```

IndexDefinition

```

struct IndexDefinition {
    std::string index_name;
    std::vector<std::string> fields; // می‌تواند چند فیلد باشد (Compound)
    IndexType type;                // SingleField, Compound, Unique
};

```

ForeignKeyDefinition

```

struct ForeignKeyDefinition {
    std::string fk_name;
    std::string local_field; // collection فیلد در این
    std::string ref_collection; // مرجع collection
    std::string ref_field; // معمولاً فیلد مرجع ("_id")
};

```

TxHandle

```

class TxHandle {
    rocksdb::Transaction* Get();
    bool IsValid();
};
// گرفته میشود از BeginTransaction()
// non-copyable, movable

```

۳. چرخه حیات

ایجاد Instance

// C++

```
#include "core/DocEngine.h"
auto engine = std::make_unique<nexora::core::DocEngine>("/var/data/nexoradb");

// بررسی سلامت
if (!engine->IsHealthy()) {
    // خطا در اتصال به RocksDB
}
```

Python (بعد از پیاده‌سازی Cython)

```
import nexoradb
engine = nexoradb.DocEngine("/var/data/nexoradb")
assert engine.is_healthy()
```

نکته مهم: سازنده در صورت باز نشدن RocksDB، `std::runtime_error` پرتاب می‌کند.

۴. مدیریت Schema و Collection

CreateCollection

```
DBResult CreateCollection(
    const std::string& collection_name,
    const std::optional<SchemaDefinition>& schema = std::nullopt
);
```

رفتار:

- Collection جدید ایجاد می‌کند

- اگر از قبل وجود داشته باشد: `success=false`
- Schema اختیاری است — بدون Schema، هر سندی پذیرفته می‌شود

// مثال C++

```
SchemaDefinition schema;
schema.strict = true;
schema.fields = {
    {"_id",      FieldType::String, true,  true },
    {"username", FieldType::String, true,  true },
    {"email",    FieldType::String, true,  true },
    {"bio",      FieldType::String, false, false},
    {"created_at", FieldType::Int64, true,  false}
};
auto r = engine->CreateCollection("users", schema);
// r.success = true
// r.data    = "Collection 'users' created"
```

مثال Cython

```
schema = nexoradb.SchemaDefinition()
schema.strict = True
schema.add_field("username", nexoradb.FieldType.String, required=True, unique=True)
result = engine.create_collection("users", schema)
```

DropCollection

```
DBResult DropCollection(const std::string& collection_name);
```

برگشت ناپذیر: تمام اسناد، Index ها، و FK های collection حذف می‌شوند.

SetSchemaValidation

```
DBResult SetSchemaValidation(
    const std::string& collection_name,
    const SchemaDefinition& schema
);
```

Schema موجود را جایگزین می‌کند. اسناد قدیمی بررسی نمی‌شوند.

```
bool CollectionExists(const std::string& collection_name) const;
```

برای بررسی سریع — بدون DBResult overhead.

۵. مدیریت Index

CreateIndex

```
DBResult CreateIndex(  
    const std::string& collection_name,  
    const IndexDefinition& index_def  
);
```

پس از ایجاد **Index**:

- تمام اسناد موجود بازنویسی می‌شوند (Rebuild)
- **FindMany** با شرط روی فیلد indexed به طور خودکار از Index استفاده می‌کند

```
// ایندکس یکتا روی username
```

```
IndexDefinition idx;  
idx.index_name = "idx_username";  
idx.fields     = {"username"};  
idx.type       = IndexType::Unique;  
engine->CreateIndex("users", idx);
```

```
// ایندکس ترکیبی (برای جستجوی پست‌های یک کاربر)
```

```
IndexDefinition idx;  
idx.index_name = "idx_author_created";  
idx.fields     = {"author_id", "created_at"};  
idx.type       = IndexType::Compound;  
engine->CreateIndex("posts", idx);
```

```
DBResult DropIndex(const std::string& collection_name, const std::string& index_name);
```

تمام entries ایندکس از RocksDB حذف می‌شوند.

۶. مدیریت Foreign Key

AddForeignKey

```
DBResult AddForeignKey(  
    const std::string& collection_name,  
    const ForeignKeyDefinition& fk_def  
);
```

// مثال - posts.author_id → users._id

```
ForeignKeyDefinition fk;  
fk.fk_name      = "fk_post_author";  
fk.local_field  = "author_id";  
fk.ref_collection = "users";  
fk.ref_field    = "_id";  
engine->AddForeignKey("posts", fk);
```

تأثیر:

- `InsertOne("posts", ...)` — مقدار `author_id` باید در `users._id` وجود داشته باشد
- `UpdateById("posts", ...)` — اگر `author_id` تغییر کند، بررسی می‌شود

// استفاده توسط GraphEngine

```
// GraphEngine در Startup:  
auto fks = engine.GetForeignKeys("posts");  
// fks[0] = {fk_name:"fk_post_author", local:"author_id", ref:"users", ref_field:"_id"}  
// در گراف است edge "Authored" از users → posts این تعریف ←
```


InsertOne

```
DBResult InsertOne(const std::string& collection_name, const std::string& bson_document)
// data = doc_id تولید/استخراج شده
```

• اگر `_id` در سند نباشد: UUID v4 تولید می‌شود

• Index Update → ذخیره → Schema Validation → FK Check

```
// بدون _id (auto-generate)
std::string bson = R("{\"username\":\"alice\",\"email\":\"alice@test.com\",\"created_at\":17000000}");
auto r = engine->InsertOne("users", bson);
std::string alice_id = r.data; // "550e8400-e29b-41d4-a716-446655440000"

// دستی _id با
std::string bson2 = R("{\"_id\":\"u_alice\",\"username\":\"alice\",\"email\":\"alice@test.com\"}");
engine->InsertOne("users", bson2);
```

InsertMany

```
DBResult InsertMany(const std::string& collection_name,
                    const std::vector<std::string>& bson_documents);
// data = JSON آرایه IDs: ["id1","id2","id3"]
```

اتمیک از طریق `WriteBatch` — یا همه موفق می‌شوند یا هیچکدام.

FindById

```
DBResult FindById(const std::string& collection_name, const std::string& doc_id);
// data = سند BSON محتوای
```

$O(1)$ — مستقیم از RocksDB.

FindMany

```
DBResult FindMany(
    const std::string& collection_name,
    const nexora::query::Condition& condition,
    uint32_t limit = 0,    // 0 = بدون محدودیت
    uint32_t skip = 0     // برای pagination
);
// data = JSON اسناد
```

نکته برای تیم **Query**: ساختار **Condition** باید از **nexora::query** namespace باشد. برای AND/OR ترکیبی، فیلدهای **and_conditions** و **or_conditions** را پر کنید.

UpdateById

```
DBResult UpdateById(
    const std::string& collection_name,
    const std::string& doc_id,
    const nexora::query::UpdateSpec& update_spec
);
// data = "0" یا "1" (موفق)
```

عملیات پشتیبانی شده در **UpdateSpec**:

عملیات	رفتار	مثال
Set	تنظیم مقدار فیلد	{Set, "bio", "Hello"}
Unset	حذف فیلد	{Unset, "temp_token", ""}
Inc	افزودن به عدد	{Inc, "likes", "1"}
Push	افزودن به آرایه	{Push, "tags", "sport"}
Pull	حذف از آرایه	{Pull, "tags", "sport"}
Rename	تغییر نام فیلد	{Rename, "old", "new"}

UpdateMany

```
DBResult UpdateMany(
    const std::string& collection_name,
    const nexora::query::Condition& condition,
    const nexora::query::UpdateSpec& update_spec
);
// data = تعداد اسناد به روز شده
```

DeleteById

```
DBResult DeleteById(const std::string& collection_name, const std::string& doc_id);
// data = "0" (پیدا نشد) (حذف شد) "1"
```

DeleteMany

```
DBResult DeleteMany(const std::string& collection_name,
    const nexora::query::Condition& condition);
// data = تعداد اسناد حذف شده
```

پیشنهاد: از Transaction استفاده کنید.

۸. Internal API برای GraphEngine

این بخش فقط برای تیم GraphEngine است. Cython layer نباید مستقیم این توابع را expose کند.

IterateCollection — مهم ترین تابع

```
void IterateCollection(
    const std::string& collection_name,
    const DocumentCallback& callback, // std::function<bool(string id, string bson)>
    uint32_t batch_size = 100
) const;
```

// کاربرد در GraphEngine::BuildGraph()

```

class GraphEngine {
    nexora::core::DocEngine& doc_engine_;
    std::unordered_map<std::string, UserNode> user_nodes_;
    std::unordered_map<std::string, PostNode> post_nodes_;

public:
    void BuildGraph() {
        // مرحله ۱: بارگذاری Users
        int64_t user_count = doc_engine_.GetCollectionSize("users");
        user_nodes_.reserve(user_count > 0 ? user_count : 10000);

        doc_engine_.IterateCollection("users",
            [this](const std::string& id, const std::string& bson) -> bool {
                UserNode node;
                node.id = id;
                node.username = ExtractField(bson, "username");
                node.email = ExtractField(bson, "email");
                user_nodes_[id] = std::move(node);
                return true; // ادامه iteration
            }
        );

        // مرحله ۲: ایجاد Posts و Edges
        doc_engine_.IterateCollection("posts",
            [this](const std::string& id, const std::string& bson) -> bool {
                std::string author_id = ExtractField(bson, "author_id");
                if (!author_id.empty() && user_nodes_.count(author_id)) {
                    // ایجاد edge: user -> post
                    AddEdge(author_id, id, EdgeType::Authored);
                }
                return true;
            }
        );
    }
};

```

نکته‌های مهم:

- `fill_cache=false` برای RocksDB — مناسب bulk scan
- اگر callback مقدار `false` برگرداند، iteration متوقف می‌شود (early exit برای partial graph)
- این تابع thread-safe نیست — در Startup (single-thread) فراخوانی شود

GetCollectionSize

```
int64_t GetCollectionSize(const std::string& collection_name) const;
// بازگشت: تعداد اسناد یا 1- در صورت خطا
```

$O(1)$ — از شمارنده داخلی RocksDB.

```
// Pre-allocate حافظه گراف
user_nodes_.reserve(engine.GetCollectionSize("users"));
post_nodes_.reserve(engine.GetCollectionSize("posts"));
```

GetDocumentRange

```
std::vector<std::pair<std::string, std::string>>
GetDocumentRange(
    const std::string& collection_name,
    const std::string& id_prefix = "",
    uint32_t max_count = 0
) const;
// بازگشت: vector<(doc_id, bson_data)>
```

// sharding یا partial graph loading استفاده برای

```
// بارگذاری 1000 کاربر اول
auto batch = engine.GetDocumentRange("users", "", 1000);
for (const auto& [id, bson] : batch) {
    graph.AddNode(id, ParseBson(bson));
}
```

GetForeignKeys — برای تشخیص edge types

```
std::vector<ForeignKeyDefinition> GetForeignKeys(const std::string& collection_name) con
```

```

void GraphEngine::DiscoverEdgeTypes() {
    auto collections = doc_engine_.ListCollections();
    for (const auto& col : collections) {
        auto fks = doc_engine_.GetForeignKeys(col);
        for (const auto& fk : fks) {
            // در گراف تعریف می‌کند edge یک نوع FK هر
            edge_types_.push_back({
                .from_collection = col,
                .to_collection   = fk.ref_collection,
                .local_field     = fk.local_field,
                .edge_name       = fk.fk_name
            });
        }
    }
}

```

۹. LookupJoin

```

JoinResult LookupJoin(
    const std::string& from_collection,
    const std::string& from_field,
    const std::string& to_collection,
    const std::string& to_field,
    const nexora::query::Condition& condition,
    uint32_t limit = 0
);

```

مثال - گرفتن پست‌های پرتعداد با اطلاعات نویسنده

```

nexora::query::Condition cond;
cond.field = "likes";
cond.op    = nexora::query::Op::GT;
cond.value = "100";

JoinResult result = engine->LookupJoin(
    "posts",    "author_id", // از posts.author_id
    "users",    "_id",       // به users._id
    cond, 20
);

if (result.success) {
    for (const auto& merged_doc : result.records) {
        // merged_doc = post + "__joined__": user_doc
    }
}

```

ساختار سند merge شده

```

{
  "_id": "post_123",
  "title": "Hello World",
  "author_id": "u_alice",
  "likes": 150,
  "__joined__": {
    "_id": "u_alice",
    "username": "alice",
    "email": "alice@test.com"
  }
}

```

۱۰. مدیریت Transaction

الگوی استفاده

// C++

```

auto tx = engine->BeginTransaction();
if (!tx) { /* handle error */ }

try {
    auto r1 = engine->InsertOneTx(*tx, "messages", msg_bson);
    if (!r1.success) throw std::runtime_error(r1.error_msg);

    auto r2 = engine->UpdateByIdTx(*tx, "users", user_id, update_spec);
    if (!r2.success) throw std::runtime_error(r2.error_msg);

    auto commit_result = engine->CommitTransaction(*tx);
    // commit_result.success = true
} catch (const std::exception& e) {
    engine->RollbackTransaction(*tx);
    // log error
}

```

Python (بعد از Cython)

```

tx = engine.begin_transaction()
try:
    engine.insert_one_tx(tx, "messages", msg_bson)
    engine.update_by_id_tx(tx, "conversations", conv_id, update_spec)
    engine.commit_transaction(tx)
except Exception as e:
    engine.rollback_transaction(tx)
    raise

```

Tx توابع

تایع	معادل بدون Tx	توضیح
InsertOneTx	InsertOne	درج در transaction
UpdateByIdTx	UpdateById	با GetForUpdate (pessimistic lock)
DeleteByIdTx	DeleteById	با GetForUpdate
FindByIdTx	FindById	با دید به uncommitted تغییرات

نکته مهم: `UpdateByIdTx` و `DeleteByIdTx` از `GetForUpdate` استفاده می‌کنند — این یعنی اگر دو transaction هم‌زمان روی یک سند کار کنند، دومی با `STATUS_BUSY` fail می‌شود.

۱۱. توابع کمکی

Count

```
DBResult Count(const std::string& collection_name, const nexora::query::Condition& condi
// data = عدد به صورت string: "42"
```

- اگر `condition.IsEmpty()` باشد: $O(1)$ از شمارنده
- در غیر این صورت: $O(n)$ full scan

Exists

```
DBResult Exists(const std::string& collection_name, const nexora::query::Condition& cond
// data = "true" یا "false"
```

بهینه‌تر از `Count` — به محض یافتن اولین نتیجه متوقف می‌شود.

ListCollections

```
std::vector<std::string> ListCollections() const;
// بازگشت: ["users", "posts", "messages", "reactions"]
```

۱۲. راهنمای تیم Query (UpdateSpec و Condition)

ساختارهای مورد نیاز

تیم query باید این فایل‌ها را در `query/` پیاده‌سازی کند:

// query/Condition.h

```
namespace nexora::query {

enum class Op : uint8_t { EQ, NEQ, GT, GTE, LT, LTE, IN, NIN, EXISTS, ALL };

struct Condition {
    std::string      field;
    Op               op    = Op::EQ;
    std::string      value;

    // ترکیبی AND/OR برای :
    std::vector<Condition> and_conditions;
    std::vector<Condition> or_conditions;

    bool IsEmpty() const noexcept;
};
```

// query/UpdateSpec.h

```
namespace nexora::query {

enum class UpdateOp : uint8_t { Set, Unset, Inc, Push, Pull, Rename };

struct UpdateOperation {
    UpdateOp    op;
    std::string field;
    std::string value;
};

struct UpdateSpec {
    std::vector<UpdateOperation> operations;
};
```

نمونه پارس در **Cython**

```
# input: {"username": "alice", "age": {"$gt": 25}}
def parse_condition(filter_dict) -> Condition:
    cond = Condition()
    if len(filter_dict) == 1:
        field, val = list(filter_dict.items())[0]
        cond.field = field
        if isinstance(val, dict):
            op_str, op_val = list(val.items())[0]
            cond.op = OP_MAP[op_str] # {"$gt": Op.GT, ...}
            cond.value = str(op_val)
        else:
            cond.op = Op.EQ
            cond.value = str(val)
    elif len(filter_dict) > 1:
        # AND ضمنی
        for k, v in filter_dict.items():
            sub = parse_condition({k: v})
            cond.and_conditions.append(sub)
    return cond
```

۱۳. راهنمای تیم GraphEngine

وابستگی به DocEngine

// graph/GraphEngine.h

```
#include "core/DocEngine.h"

class GraphEngine {
public:
    // DocEngine باید از بیرون inject شود
    explicit GraphEngine(nexora::core::DocEngine& engine);

    // فراخوانی می‌شود در Startup
    void BuildGraph();

    // گراف API
    std::vector<std::string> GetFollowers(const std::string& user_id);
    std::vector<std::string> GetSuggestedPosts(const std::string& user_id, int limit);
    // ...

private:
    nexora::core::DocEngine& doc_engine_;

    // ساختارهای in-memory
    std::unordered_map<std::string, UserNode> user_nodes_;
    std::unordered_map<std::string, PostNode> post_nodes_;

    // adjacency lists
    std::unordered_map<std::string, std::vector<std::string>> followers_;
    std::unordered_map<std::string, std::vector<std::string>> following_;
};
```

توالی Startup

```
main() یا Python import:
1. DocEngine engine("/data/nexoradb")
2. engine.IsHealthy() ← بررسی
3. GraphEngine graph(engine)
4. graph.BuildGraph()
   ├── engine.GetCollectionSize("users")      ← pre-allocate
   ├── engine.IterateCollection("users", ...) ← بارگذاری nodes
   ├── engine.IterateCollection("posts", ...) ← بارگذاری edges
   └── engine.GetForeignKeys("posts")          ← تشخیص edge types
5. ✓ سرویس آماده است
```

نکته مهم برای همگام‌سازی

DocEngine **thread-safe** نیست. یک راه حل:

```

class NexoraDB {
    std::mutex          engine_mutex_;
    DocEngine           doc_engine_;
    GraphEngine         graph_engine_;

    // در Startup: single-threaded BuildGraph
    // در Runtime: هر write باید graph را invalidate کند
    DBResult InsertWithGraphUpdate(const std::string& col, const std::string& bson) {
        std::lock_guard<std::mutex> lock(engine_mutex_);
        auto r = doc_engine_.InsertOne(col, bson);
        if (r.success) graph_engine_.OnDocumentInserted(col, r.data, bson);
        return r;
    }
};

```

۱۴. راهنمای تیم Cython/Python Bindings

فایل‌های مورد نیاز

```

bindings/python/
├─ nexoradb.pxd    # تعاریف C++ (declarations)
└─ nexoradb.pyx    # پیاده‌سازی Python wrapper

```

نمونه nexoradb.pxd

```

# distutils: language = c++
from libcpp.string cimport string
from libcpp.vector cimport vector
from libcpp cimport bool
from libcpp.memory cimport unique_ptr

cdef extern from "core/DocEngine.h" namespace "nexora::core":

    cdef struct DBResult:
        bool success
        string data
        string error_msg

    cdef cppclass DocEngine:
        DocEngine(const string& db_path) except +
        bool IsHealthy() const
        DBResult CreateCollection(const string&, ...)
        DBResult InsertOne(const string&, const string&)
        DBResult FindById(const string&, const string&)
        # سایر توابع ...

```

نمونه nexoradb.pyx

```

# distutils: language = c++
# cython: language_level=3

cimport nexoradb # از .pxd
from cpython.bytes cimport PyBytes_FromStringAndSize

cdef class PyDocEngine:
    cdef nexoradb.DocEngine* _engine

    def __cinit__(self, str db_path):
        self._engine = new nexoradb.DocEngine(db_path.encode('utf-8'))

    def __dealloc__(self):
        if self._engine != NULL:
            del self._engine

    def is_healthy(self) -> bool:
        return self._engine.IsHealthy()

    def insert_one(self, str collection, bytes bson_doc) -> dict:
        cdef nexoradb.DBResult result = self._engine.InsertOne(
            collection.encode('utf-8'), bson_doc
        )
        return {
            'success': result.success,
            'data': result.data.decode('utf-8'),
            'error_msg': result.error_msg.decode('utf-8')
        }

    def find_by_id(self, str collection, str doc_id) -> dict:
        cdef nexoradb.DBResult result = self._engine.FindById(
            collection.encode('utf-8'),
            doc_id.encode('utf-8')
        )
        return {
            'success': result.success,
            'data': result.data.decode('utf-8'),
            'error_msg': result.error_msg.decode('utf-8')
        }

# سایر توابع ...

```

```

from setuptools import setup, Extension
from Cython.Build import cythonize
import os

ext = Extension(
    "nexoradb",
    sources=["bindings/python/nexoradb.pyx", "core/DocEngine.cpp"],
    include_dirs=[".", "/usr/include"],
    libraries=["rocksdb"],
    extra_compile_args=["-std=c++20", "-O2", "-fPIC"],
    language="c++"
)

setup(
    name="nexoradb",
    version="0.1.0",
    ext_modules=cythonize([ext], compiler_directives={'language_level': "3"}),
)

```

۱۵. فرمت کلیدهای RocksDB

محتوا	فرمت کامل	Prefix
سند BSON باینری	<code>data:{collection}:{doc_id}</code>	<code>data:</code>
SchemaDefinition serialize شده	<code>meta:col:{collection_name}</code>	<code>meta:col:</code>
IndexDefinition serialize شده	<code>meta:idx:{collection}:{index_name}</code>	<code>meta:idx:</code>
ForeignKeyDefinition serialize شده	<code>meta:fk:{collection}:{fk_name}</code>	<code>meta:fk:</code>
doc_id (برای index lookup)	<code>idx:{collection}:{field}:{value}:{doc_id}</code>	<code>idx:</code>
شمارنده اسناد (integer string)	<code>seq:{collection}</code>	<code>seq:</code>

۱۶. محدودیت‌ها و TODO های آینده

نسخه MVP (فعلی)

موضوع	وضعیت	توضیح
Thread Safety		از locking خارجی استفاده کنید
BSON Library		پیاده‌سازی ساده JSON-like — باید با libbson جایگزین شود
FindMany در Index Scan		هنوز full scan است — در نسخه بعدی index scan اضافه می‌شود
Push/Pull/Rename در UpdateSpec		پیاده‌سازی کامل نشده
Cascading Delete برای FK		در MVP بررسی FK فقط در Insert/Update است
Compression		Snappy فعال است
Transactions		ACID از طریق RocksDB TransactionDB
Schema Validation		required فیلدها بررسی می‌شوند

نسخه‌های بعدی

TODO v0.2:

- [] FindMany در Index Scan پیاده‌سازی
- [] واقعی BSON برای libbson جایگزینی با
- [] sessions برای TTL (Time-To-Live) اضافه کردن
- [] Aggregate Pipeline (Count, Sum, Group)
- [] Full-text Search Index
- [] GraphEngine برای Read-only Snapshot

مستندات توسط تیم NexoraDB Core تهیه شده است — آخرین بهروزرسانی: MVP v0.1.0